

Tetris - Reinforcement Learning²

Paul Kehnel

ga65coy@mytum.de

Sebastian Fellner

ga96kug@mytum.de

Simon Speth

ga96tov@mytum.de

Benedikt Wiberg

b.wiberg@tum.de

Project Proposal

1. Introduction

Tetris is a good example task for reinforcement learning. It is an interesting problem, because it is quite hard in a mathematical sense since "Finding the optimal strategy is NP-hard even if the sequence of tetrominoes is known in advance." [4] But since superhuman performance is still easily achievable, we want to approach "Blind-Tetris". Our inspiration was this [Tetris Video](#) we found on Youtube. We first want to train a reinforcement learning agent to play tetris. Afterwards we iteratively limit the input information about the game state to complicate the problem. If we have time, we would like to try the different advancements made in reinforcement learning described in [2] to improve our solution.

1.1. Related Works

- **Learning Tetris using the noisy cross-entropy method:** This is one of the older approaches. They apply noise for preventing early convergence of the cross-entropy method, by that optimising and outperforming previous RL algorithms. [4]
- **Playing atari with deep reinforcement learning:** In the field of reinforcement learning, this paper by Deepmind is currently one of the most impressive works and can be seen as fundamental part for further improvements. [3]
- **Rainbow: Combining Improvements in Deep Reinforcement Learning:** This is currently state of the art, by combining different enhancements for DQN. This is crucial for selecting well fitting enhancements for our problem. [2]

2. Dataset

Since we are using reinforcement learning there is no need for collecting data beforehand. If we want to try supervised training we may need to record ourselves playing the game for a few hours. We may need to implement our own version of tetris fitted for our needs or we will use one of the numerous [Open Source](#) implementations.

Data collected during runtime will most likely not be independent and identically distributed, as a result we will have to apply some additional processing steps, like experience replay. Also feedback-loops could appear. Furthermore because we are designing the reward function, our data will be automatically labelled.

Finally our data input will consist of matrix describing the gamestate and the next piece. Logical actions consisting of rotation and placement of the next piece will be the output.

3. Methodology

After we fitted an implementation of tetris to our needs, we will first train a Deep Q-Network from scratch on the normal tetris task. This will set our baseline. Afterwards we will remove gamestate information consisting of n bottom rows. We will add LSTM-Cells to train our network to implicitly learn the removed gamestate. If we cant match our baseline, we will try leveraging the advancements described in [2] and/or use hybrid learning techniques[1]. Because implementing tetris on a GPU is not an easy task, we will be limited to train on a CPU.

4. Outcome

We want to see how much information the agent can retain about the gamestate. Our desired output would be an agent that can play tetris by just getting the next piece as input.

References

- [1] J. Henderson, O. Lemon, and K. Georgila. Hybrid reinforcement/supervised learning for dialogue policies from communicator data. In *IJCAI workshop on knowledge and reasoning in practical dialogue systems*, pages 68–75, 2005. 1
- [2] M. Hessel, J. Modayil, H. Van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver. Rainbow: Combining improvements in deep reinforcement learning. *arXiv preprint arXiv:1710.02298*, 2017. 1
- [3] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013. 1

- [4] I. Szita and A. Lörincz. Learning tetris using the noisy cross-entropy method. *Learning*, 18(12), 2006. [1](#)